A Timing and Power-based Side-channel protected HQC Realization on Cortex-M4

Omitted for blind review

Omitted for blind review

Abstract. In this paper we show the first constant-time and fully masked implementation of HQC targeting Cortex-M4 microcontrollers. Our proposal for a masked implementation is constant-time and has a less-than linear overhead factor in the number of shares, thanks to appropriate exploitations of the Boolean linearity of the components of HQC. We demostrate the efficiency of our implementation and its constant-time behavior concretely, through taking clock-cycle level timing and electromagnetic emission measures on a commercial grade Cortex-M4, the current NIST reference microcontroller platform.

Keywords: Post-quantum cryptography, code-based cryptosystems, constant time implementation, masking

1 Introduction

Post-quantum cryptography (PQC), also known as quantum-resistant cryptography, aims at defining new public-key cryptographic schemes based on mathematical problems proven to be computationally hard against adversaries equipped with both classical and quantum computers. Recent advancements on quantum computer technology have witnessed a steep increase in scientific and engineering activities to deliver new solutions to some of the most pressing scientific and societal challenges [5]. However, from a cybersecurity standpoint, the availability of a quantum computer with a sufficiently large number of qubits, would leave most of the current protocols used to guarantee the confidentiality and authenticity of communications and data storage vulnerable to cryptanalytic techniques based on Shor [30]-like quantum algorithms that subvert the security guarantees of the underlying public key cryptosystems.

Since 2017, several institutional and governmental bodies promoted the analysis of quantum-secure systems, but most stakeholders track the efforts of the PQC standardization contest promoted by the U.S. National Institute of Standard (NIST). Through two rounds of this evaluation process, NIST selected seven finalists and eight alternate candidates, which were then thinned down to two standard candidates and four alternates for a final round of cryptanalysis and possible standardization. The current standardization-elected candidate Key Encapsulation Method (KEM) is Kyber [21], a lattice-based cryptosystem, while NIST selected four candidates relying on different computationally

hard problems, in order to obtain resilience against potentially disruptive attacks on lattice-based cryptosystems. Among the four candidates, a polynomial-time cryptanalytic attack eliminated SIKE, leaving only three KEMs which are based based on the hardness of the syndrome decoding problem, i.e., Classic McEliece [4], BIKE [2] and HQC [22]. Among these candidates, only HQC can exhibit a security proof based on a polynomial reduction of the solution of a computationally hard problem to the breaking of the cryptosystem. In particular, any successful break of HQC implies a faster way to solve the syndrome decoding problem on a random quasi-cyclic code of rate $\frac{1}{2}$. By contrast, Classic McEliece relies on an additional assumption, namely the indistinguishability of a hidden Goppa code from a random code, and BIKE relies on the indistinguishability of a hidden QC-MDPC code from a random quasi cyclic code. While both arguments are currently unchallenged, there is no proof that they are indeed corresponding to computationally hard problems.

Hamming Quasi-Cyclic (HQC) [22,23] is a code-based IND-CCA secure KEM whose security relies on the syndrome decoding problem for quasi cyclic codes. It is obtained by applying the Hofheinz-Hövelmanns-Kiltz (HHK) transform [16] to a IND-CPA construction, which is also denoted as HQC.PKE. Given the sound security record of HQC, much attention has been devoted to its side-channel cryptanalysis. Most of the independent analyses focused mainly on power analysis or on extracting electromagnetic traces [13,12,26,28]; nonetheless, in [14] the authors described a timing-based key-recovery attack against HQC, based on an implementation weakness in the step that used a procedure called *rejection sampling* during randomness generation. As a countermeasure, a more careful implementation of the rejection sampling step is currently employed.

To date, no timing and power- or Electromagnetic(EM)-based side-channel secure implementation of HQC, targeted to embedded systems, has been proposed, despite the real threat set by these attacks. Moreover, we also note that most of the literature focused on timing side-channel attacks aims at key retrieval, while it is also possible to perform a message-retrieval attack against the encryption primitive, in case the operations in the said encryption are not constant time. Indeed, the attack mechanism mimics the common attacks on modular exponentiation, as the non-constant time polynomial multiplication techniques, such as the one in the reference code by HQC, have a loop whose body contains branches depending on the values of a secret operand. Since, in the HQC.PKE encryption primitive, at least one of such operands is secret (as it is the plaintext message to be encrypted), providing constant time multiplication algorithms is needed to secure the encryption primitive. Furthermore, as the HHK transform requires a execution of the HQC.PKE encryption function during the computation of the HQC KEM decryption, the requirement for constant time multiplication implementations is further strengthened.

The current state of the software HQC implementations tailored to the ARM Cortex-M4 microcontroller (chosen by NIST as the reference platform for microcontroller benchmarks) includes only the one in PQM4 [18], which is a public software project collecting the implementation of all NIST fourth round candi-

dates tailored for the Cortex M4 CPUs. In the PQM4 project, the latest version of the HQC implementation is directly derived from the third round reference one, without further optimizations. As a consequence, the implementation is vulnerable to the timing side-channel attack by [14]. Furthermore, no protection against power-based side-channel attacks is present.

Our Contribution. In this paper, we present the first fully constant-time and power/EM side-channel protected software implementation of HQC.KEM, tailored to Cortex-M4 microcontrollers. We provide an experimental validation of the security of our proposal via a battery of statistical t-tests, both for the EM-emissions and execution time side-channels. We evaluate the timing side-channel immunity while varying the CPU frequency and memory latencies, to highlight the stability of our solution across different memory access latencies (which depend on the CPU frequency). Our results show that employing a FFT-based multiplication algorithm is the optimal choice for constant-time multiplications on Cortex-M4, in contrast with classic divide-et-impera techniques (e.g., Karatsuba) which provide a better tradeoff for x86_64 computing platforms. The new algorithmic strategy we introduce to hinder power/EM side-channel attacks against HQC.KEM software implementations achieves a lower than linear performance overhead in protecting the HQC.KEM Decapsulation primitive, currently a unique and welcome feature among side-channel protected implementations of post-quantum KEMs. Our approach is specifically tailored to the decapsulation of HQC, while applying an n out of n share-splitting strategy to Key-generation and Encapsulation primitives with a linear performance overhead.

2 Background

A Key Encapsulation Mechanism (KEM) is an asymmetric encryption technique designed to generate both a randomly chosen symmetric key k_s and its encrypted version c, denoted as *encapsulated key*. Considering an insecure communication channel, the encapsulation c serves as a secure delivery of the secret key, while k_s is meant to be used for the subsequent symmetric encryption of data exchanged between the endpoints of the communication. Unlike public key encryption primitives (PKEs), such as either RSA [25] or the ElGamal [10] encryption algorithms, to prepare a proper plaintext there is no need for a criterion to map bitstrings into elements of an algebraic structure, Furthermore, since the bulk of the data is encrypted with the conveyed symmetric key k_s, they can be received as a stream, without the need to buffer the whole ciphertext before decryption. From a security standpoint, the only (weak) requirement on the symmetric-key cipher used for subsequent data bulk encryption is that it exhibits a stream-based mode of operation (e.g., CTR - CounTeR mode) that is secure in case k_s is employed only once per data exchange session.

Post-quantum cryptographic primitives define PKEs that are only weakly secure, due to the presence of probabilistic correctness errors or because they are proven to exhibit only the computational property of either *indistinguishability of ciphertexts under chosen-plaintext attack* (IND-CPA) or *one-wayness under chosen-plaintext attack* (OW-CPA). A PKE benefits from OW-CPA if an

adversary equipped with a public key and the corresponding encryption of a random plaintext, is computationally unable to derive the entire plaintext. It is well known that IND-CPA implies OW-CPA, because a PKE benefits from the IND-CPA property when an adversary is unable to distinguish the encryptions of two messages of his choice; more precisely, given the public key and the corresponding encryption of one of two attacker-chosen plaintexts, the attacker is able to determine which of the two messages was encrypted only with negligible success probability. To guarantee security in both the Random Oracle Model (ROM) and the Quantum Random Oracle Model (QROM), every post-quantum cryptographic scheme makes use of a variant of the so-called Fujisaki-Okamoto (FO) transformation [9], introduced for the first time in 1999. Nowadays, a newer fine-grained modular toolkit of improved FO transformations [16] is employed for turning weakly secure PKE schemes into strongly secure ones, i.e., schemes benefiting from the property of *indistinguishability of ciphertexts under adaptive chosen- ciphertext attack* (IND-CCA), i.e., an adversary is unable to distinguish the encryptions of two messages of his choice even if he is allowed access to a decryption oracle before and after seeing the (randomly selected) ciphertext of one of the two messages he has chosen (honoring the constraint of not querying the oracle with the challenge ciphertext itself).

2.1 HQC cryptosystem

The first code-based PKE was introduced by McEliece in 1978 and is based on the assumption that the underlying forward error-correction code, a Goppa code, used is indistinguishable from a random one. Although the original McEliece cryptosystem remains secure to this day as demonstrated by the highly optimized Classic-McEliece cryptoscheme [4], the storage requirements for its public key, which is not smaller than 0.25 MiB for 128-bit security, is deemed too large for many application scenarios. Attempts to reduce the key size of the McEliece PKE by using codes more structured than Goppa codes have been all cryptanalyzed. Therefore, the authors of the IND-CCA *Hamming Quasi-Cyclic* (HQC) code-based post-quantum KEM proposed a novel approach for the design of their PKE, which combines two different types of codes: *(i)* An $[n, k]$ code $\mathcal{C}$ defined as the concatenation of (*shortened*) Reed-Müller (RM) and (*duplicated*) Reed-Solomon (RS) codes, for which an efficient decoding algorithm is known, with a fixed, publicly known generator matrix $\mathbf{G} \in \mathbb{F}_2^{k\times n}$ ensuring the capability to correct up to δ errors (i.e., bit-flips) in a n-bit codeword obtained from a message $\mathbf{m} \in \mathbb{F}_2^k$ encoded as a (row) binary vector, as $\mathbf{mG}$. *(ii)* A random double-circulant $[2n, n]$ code with a publicly known parity-check matrix $\mathbf{H}\in\mathbb{F}_2^{n\times 2n}$, $\mathbf{H}=[\mathbf{I_n}|\mathbf{rot}(\mathbf{h})]$, where $\mathbf{I_n}$ is the identity matrix and $\mathbf{rot}(\mathbf{h})$ is the circulant $n \times n$ matrix induced by the $1 \times n$ binary vector $\mathbf{h}$, i.e., the matrix obtained from the juxtaposition of $\mathbf{h}$ as the first column and the subsequent columns obtained through a vertical rotation, by one cell, of the previous one. The coefficients of the $1 \times n$ binary vector $\mathbf{h} \in \mathbb{F}_2^n$ are seen as the coefficients of a randomly chosen binary polynomial in $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$, where n is a prime number such that $\mathtt{ord}_2(n) = n - 1$, i.e., $2^{n-1} = 1 \bmod n$. The design rationale is

that the double-circulant code is going to be used to add some "noise" (i.e., bit-flips) to the codewords of the inner RM-RS code, which in turn will be partially removed through the use of the secret key. The leftover noise plus an additional uncorrelated noise term will be decoded by the code $\mathcal{C}$, following the same generic steps of the ElGamal cryptosystem [10] and adapting to quasi-cyclic (QC) codes the approach first introduced in [1]. In such a way, HQC can exhibit significantly smaller keys than the Classic-McEliece KEM, while still providing the same computational security guarantees. Indeed, for a security level of 128-bit, the public key size is around 2 kiB vs 256 kiB of the Classic-McEliece scheme. Specifically, the PKE underlying the HQC design is defined by the following three algorithms.

$\mathsf{Keygen}()$: samples $\mathbf{h} \xleftarrow{\$} \mathcal{R}$, $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$, and outputs the private and public keypair as: $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$ and $\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s} = \mathbf{H}[\mathbf{x}|\mathbf{y}]^T = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$. $\mathcal{R}_\omega$ denotes the set of binary polynomials (i.e., vectors in $\mathbb{F}_2^n$) with Hamming Weight ω (i.e., the number of non-zero coefficients), while $\cdot$ is the circular convolution of the two sequences of coefficients of the factors (each of which is considered as either a binary vector or a binary polynomial); e.g., given $\mathbf{a}, \mathbf{b}, \mathbf{c} = \mathbf{a} \cdot \mathbf{b} \in \mathbb{F}_2^n$, it holds that $\mathbf{a}=(a_0, \ldots, a_i, \ldots, a_{n-1})$, $\mathbf{b}=(b_0, \ldots, b_j, \ldots, b_{n-1})$ and $\mathbf{c} = \mathbf{a} \cdot \mathbf{b} = \sum_{\substack{k=0 \\ i+j \equiv k \bmod n}}^{n-1} a_i b_j$.

$\mathsf{Encrypt}(\mathcal{K}_{pub}, \mathbf{m})$: takes as input the public key, $\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$, samples an ephemeral $1 \times n$ error vector $\mathbf{e}$ from the subset of $\mathcal{R}$ that includes items with Hamming Weight ω_e, i.e., $\mathbf{e} \xleftarrow{\$} \mathcal{R}_{\omega_e}$, samples the ephemeral vector $r = [\mathbf{r_1}|\mathbf{r_2}] \in \mathbb{F}_2^{2n}$ in such a way that each pair $(\mathbf{r_1}, \mathbf{r_2})$ is picked from the Cartesian product of two instances of the subset of $\mathcal{R}$ that includes items with Hamming Weight ω_r, i.e., $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$, and output a ciphertext $\mathbf{c}$ which, in turn, is composed by the pair $(\mathbf{u}, \mathbf{v})$, where:

$$\mathbf{u} = \mathbf{r_1} + \mathbf{h} \cdot \mathbf{r_2}, \text{ and } \mathbf{v} = \mathbf{mG} + \mathbf{s} \cdot \mathbf{r_2} + \mathbf{e}$$

$\mathsf{Decrypt}(\mathcal{K}_{pri}, (\mathbf{u}, \mathbf{v}))$: takes as input the private key, $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$ and a ciphertext $(\mathbf{u}, \mathbf{v}) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$, and outputs the original plaintext message through computing $\mathbf{m} = \mathcal{C}.\mathsf{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$.

The computational security of recovering either the secret-key or the plaintext of an instance of the HQC.PKE scheme is computationally equivalent to solving the syndrome decoding problem on random quasi-cyclic codes with length n and dimension k [23]. While random quasi-cyclic codes are not proven to yield an NP-complete and hard to approximate syndrome decoding problems, it is largely believed that, in case n is chosen as a prime such that $\mathtt{ord}(2)=n-1$, no speedup can be achieved with respect to the solution of the syndrome decoding problem on random codes with the same length, when the error generating the syndrome is half of the code distance expected for a random code [23].

Thus, the best classical and quantum cryptanalytic techniques that are currently known to perform a key- or a message-recovery attack against the cryptosystem have been considered to design the parameters sets shown in Table 1.

Table 1: HQC Parameter sets. n: public code length, n_1: Reed–Solomon code length, n_2: Reed–Müller code length. ω: weight of the n-bit binary vectors $\mathbf{x}, \mathbf{y}$, composing the secret key. $\omega_r = \omega_e$: weight of the n-bit binary vectors used in the encryption. $[n, k, d]$ for Reed–Solomon and Reed–Müller codes are the length, dimension, and minimum distance of the codes, respectively. The multiplicity of duplicated Reed–Müller code is 3, 5, 5 for HQC-128, HQC-192, HQC-256. The probability of a decapsulation error (i.e., of a correctness error of the system) is specified as the decoding failure rate (DFR) of the public code

NIST Category	**Instance**	**Plaintex size (bit)**	**Reed-Solomon**	**Reed-Müller**	**n**	ω	$\omega_r = \omega_e$	**DFR**
1	`hqc-128`	128	$[46, 16, 15]$	$[384, 8, 192]$	$17,669$	66	75	2^{-128}
3	`hqc-192`	192	$[56, 24, 16]$	$[640, 8, 320]$	$35,851$	100	114	2^{-192}
5	`hqc-256`	256	$[90, 32, 29]$	$[640, 8, 320]$	$57,637$	131	149	2^{-256}

In particular, the NIST call defines the security levels for which parameter sets should be designed stating that the resulting computational effort to break the cryptoscheme should match the one to break both classical and quantum realizations of AES-128, AES-192, and AES-256, respectively. The sets of parameters of a post-quantum cryptosystem that match the security desiderata are labeled by NIST as Category 1, Category 3 and Category 5, respectively, while the authors of HQC.KEM named the corresponding instances of their proposal as `hqc-128`, `hqc-192` and `hqc-256`. Finally, the most up-to-date specification of HQC reports a public code that is defined as the concatenation of Reed-Solomon (outer) and Reed-Müller (inner) codes, with codeword length equal to n_1 and n_2, respectively, while the overall codeword length n is taken as the smallest prime number greater than $n_1 n_2$ such that $\texttt{ord}_n(2)=n-1$. Thus, the public generator matrix $\mathbf{G}$ has dimension $k \times n$, where k coincides with the bitsize of the plaintext message $\mathbf{m}$. Furthermore, the random code with parity-check matrix $\mathbf{H}$ is instantiated by randomly sampling the vector $\mathbf{h}$ from $\mathbb{F}_2^n$.

3 Analysis of HQC Operations

The HQC.KEM is obtained starting from the HQC.PKE, augmenting it with components which do not involve polynomial arithmetic operations. In particular, the HQC.KEM employs the SHAKE extendable output function [8] (i.e., a function mapping an arbitrary-length bit string to a bitstring with a user chosen length, which is indistinguishable from a randomly selected string; in other words a PRNG algorithm with a seed of arbitrary length) to realize the variant of the FO transform [16]. Since the constant time and side channel protection of SHAKE have been successfully tackled in the literature, we consider them out of scope for this work, and employ the existing results. Except for SHAKE, the HQC.KEM algorithmic components are the same of the HQC.PKE, namely polynomial addition and multiplication in $\mathcal{R}$, pseudorandom bit generation via SHAKE and encoding/decoding for the concatenated

Reed-Müller Reed-Solomon code. Considering the fact that binary polynomial additions are straightforward to compute in constant time, and protect with respect to side channel attacks, we focus on polynomial multiplications and encoding/decoding. The pseudorandom number generation component is effectively and efficiently protected in the same way the HQC.KEM SHAKE element is.

Concerning the encoding/decoding, we devise an algorithmic strategy to avoid the requirement for a power based side channel attack protected decoder, exploiting the linearity of the RM-RS codes, while no timing analysis concerns are present thanks to their constant time implementation being the current one at the state of the art. In the following, we describe in detail the approaches to a timing-side channel protected polynomial multiplication, while deferring the protection of the said multiplication against power analysis attacks to the description of our algorithmic strategy for the protection of the entire HQC.PKE.

3.1 Polynomial Multiplication Approaches

HQC.PKE uses polynomial multiplications in $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ in each of its primitives. Referring to the definition of the HQC.PKE primitives and to Table 1, Keygen() requires one multiplication between elements in $\mathcal{R}$, where the polynomial employed as a second factor has only ω of non-null coefficients, with $\omega \ll n$, i.e., $\mathbf{h} \cdot \mathbf{y}$, with $\mathbf{h} \in \mathcal{R}$, $\mathbf{y} \in \mathcal{R}_\omega$. Thus, in any software implementation, the first factor (e.g., $\mathbf{h}$) is stored as a *dense* polynomial, i.e., a sequence of n binary coefficients, while the second one ($\mathbf{y}$) is stored as a *sparse* polynomial, i.e., as a sequence of integer numbers, each of which is intended as the exponent of a non-null monomial ($x^i, 0 \leq i \leq n-1$). Encrypt() requires two multiplications in $\mathcal{R}$ to compute the ciphertext, in each of which the first factor is a dense polynomial and the second factor is a sparse polynomial with a number of coefficients equal to ω_r. Finally, Decrypt() requires one multiplication in $\mathcal{R}$, where the first factor is a dense polynomial ($\mathbf{u}$) and the second one is a sparse polynomial ($\mathbf{y}$) with ω non-null coefficients.

Denoting as `wsize` the bitlength of the word of the CPU employed as a target computational platform, the data structure for storing a dense polynomial $b(x) \in \mathcal{R}$ is an array $B = (B_{t-1}, \ldots, B_j, \ldots, B_0)_{\mathtt{w}}$ of $t = \lceil \frac{n}{\mathtt{wsize}} \rceil$ cells, where each cell B_j stores a sequence of adjacent binary coefficients in $b(x)$ that fits in a CPU word. Equivalently, the same polynomial can be thought of as a multiple precision unsigned integer encoded in t CPU words, with each digit (limb) $0 \leq B_j < \mathtt{w}$, $0 \leq j \leq t-1$, meant to follow the usual positional notation of a number in radix $\mathtt{w} = 2^{\mathtt{wsize}}$. A sparse polynomial $a(x) \in \mathcal{R}$ is an array $A = (A_{\omega-1}, \ldots, A_i, \ldots, A_0)$, with $\omega \ll n$ cells, such that each cell $A_i = a$ is an unsigned integer that encodes the exponent of a non-null monomial $x^a \in \mathcal{R}$ included in $a(x)$, where the value $0 \leq a \leq n-1$ is encoded with at most $m = \lceil \log_2(n) \rceil$ bits. The product $c(x) = b(x) \cdot x^{A_i} \bmod (x^n - 1)$ between a dense polynomial $b(x)$ and a monomial with $A_i = a$ must be equivalent to the computation of a circular left-shift of the coefficients of $b(x)$, for a positions across their original n-bit positions (also denoted as $\lll_n a$).

Algorithm 1: Constant-time Sparse-to-dense Polynom. Multiplication

Input: n: maximum number of bits for a polynomial in $\mathcal{R}{=}\mathbb{F}_2[x]/(x^n{-}1)$.
`wsize`: bitlength of a CPU word, $\mathtt{w} = 2^{\mathtt{wsize}}$ radix employed to represent in positional notation a long integer or the sequence of coefficients of a binary-polynomial.
$A = (A_{\omega-1}, \ldots, A_i, \ldots, A_0)$, represents $a(x) \in \mathcal{R}$, where $A_i = a$ is an unsigned integer that encodes the exponent of a non-null monomial $x^a \in \mathcal{R}$, $0 \leq a \leq n-1$, $\omega < n$, $a = (a_{m-1}, \ldots, a_0)_{\mathtt{bin}}$, $a_i \in \{0, 1\}$, $m = \lceil \log_2(n) \rceil$.
$B = (B_{t-1}, \ldots, B_j, \ldots, B_0)_{\mathtt{w}}$, $t = \lceil \frac{n}{\mathtt{wsize}} \rceil$, represents $b(x) \in \mathcal{R}$, where a limb B_j stores a sequence of binary coefficients fitting in a CPU word having `wsize` bits (i.e., $0{\leq}B_j{<}\mathtt{w}$), $0{\leq}j{\leq}t{-}1$.

Output: $C = (C_{t-1}, \ldots, C_k, \ldots, C_0)_{\mathtt{w}}$, represents $c(x) = x^a \cdot b(x) \in \mathcal{R}$, where each limb C_k stores a sequence of binary coefficients fitting into a CPU word (i.e., $0{\leq}C_k{<}\mathtt{w}$), $0{\leq}k{\leq}t{-}1$.

```
1 for i ← 0 to ω − 1 do
2 |   a ← A_i; // value with m=⌈log2(n)⌉ bits: (a_{m−1}, ..., m_0)_bin
3 |   C ← ( B ≪_n (a mod wsize) ); // computation of a t-word array
4 |   for j ← log2(wsize) to (m − 1) do
5 |   |   tmp ← ( B ≪_n 2^j ); // computation of a t-word array
6 |   |   mask ← 0 − (a_j = 1); // single word: all its bits are 1 or 0
7 |   |   for k ← 0 to (t − 1) do
8 |   |   |   C_k ← C_k ⊕ ((C_k ∧ mask) ∨ (tmp_k ∧ mask));
9 return C;
```

3.2 Sparse-to-dense Multiplication Algorithm

In the third round of the NIST standardization process, the authors of HQC proposed, as a reference implementation, only a schoolbook non-constant time polynomial multiplication algorithm using a minor variant of the schoolbook *Shift-and-Add* method, exploiting the sparsity of one of the two operands. The approach is summarized as $c(x) = b(x) \cdot a(x) = b(x)(x^{A_{\omega-1}} \oplus \cdots \oplus x^{A_i} \oplus \ldots)$ $= b(x)x^{A_{\omega-1}} \oplus \cdots \oplus b(x)x^{A_i} \cdots \oplus b(x)x^{A_0}$, where $A_{\omega-1}, \ldots, A_0 \in \{0, \ldots, n-1\}$ and each multiplication by a monomial is computed as a circular left-shift of the coefficients of the first polynomial $b(x)$ for A_i positions across its original n bit positions. The pseudo-code of the method is in Algorithm 5, Appendix 5, while its worst-case computational complexity is $O(\omega(2t))$ bit operations.

The major drawback of the schoolbook *Shift-and-Add* method lies in the variable-time computation reported on Alg. 5, **line** 3, which reveals on the timing side-channel, the sensitive value of each monomial coefficient A_i of the sparse polynomial factor. Indeed, the computation time of a circular left-shift for a positions across the original n bit positions of the first array is a multiple of the execution time required to compute $(B \lll_n 1)$. A constant-time realization of the Shift-and-Add strategy is shown in Algorithm 1. The rationale is still to split the computation as the iterated sum of the dense polynomial multiplied by a monomial one. However, instead of executing in variable-time every single monomial multiplication, it is possible to realize it as a fixed sequence of circular

left-shift operations followed by arithmetic predicates for the conditional committment of each of them. Specifically, considering $a{=}(a_{m-1},\ldots,a_j,\ldots,a_0)_{\texttt{bin}}$, $0{\leq}j{<}m$, $a_j{\in}\{0,1\}$, $m{=}\lceil\log_2(n)\rceil$, the multiplication $c(x) = b(x){\cdot}x^a \bmod (x^n{-}1)$ is decomposed as:

$$c(x) = b(x)\cdot\left(x^{a_{m-1}2^{m-1}}\cdots x^{a_j 2^{m-1}}\cdots x^{a_0}\right) \bmod (x^n-1) =$$
$$= b(x)\cdot\left(x^{a_{m-1}2^{m-1}}\cdots x^{a_{\log_2(\texttt{wsize})}2^{\log_2(\texttt{wsize})}}\cdots x^{a \bmod \texttt{wsize}}\right) \bmod (x^n-1),$$

observing that `wsize` is a power of two (for any CPU), thus $a = (a_{m-1},\ldots,a_0)_{\texttt{bin}}$ $= (a_{m-1},\ldots,a_{\log_2(\texttt{wsize})})_{\texttt{bin}}$ concatenated with the binary string encoding (a mod `wsize`), while the execution of $b(x)\cdot x^{a \bmod \texttt{wsize}}$ is amenable to be realized through a constant amount of $t = \lceil\frac{n}{\texttt{wsize}}\rceil$ circular left-shifts. These observations, together with the arithmetic test to establish whether any a_j is asserted or not, and a conditional update of the final result (see Alg. 1: **lines** 4–8), allow to realize a constant-time version of the Shift-and-Add method with a computational cost that amounts to $\Theta\left(\omega\,(2t)\,m + \omega\,t + \omega\,m\right)$ bit operations.

3.3 Dense-to-dense Multiplication Algorithms

Another approach to constant-time polynomial multiplication is to give up the savings offered by the sparse representation of one of the factors and represent both factors as dense polynomials. Indeed, the execution of algorithmic strategies as the schoolbook multiplication (recommended for operands encoded with up to $\approx$ 300 bits), the Karatsuba-, Toom-Cook- or Fast-Fourier-Transform-based (FFT) multiplications (recommended for operands with bitlength up to $\approx 10^4$, from $\approx 10^4$ to $\approx 6\cdot 10^4$, and $\gtrapprox 6\cdot 10^4$, respectively) exhibit no information leakage on the timing side-channel.

HQC Approach. In the fourth round of the NIST standardization process, given the size of the polynomials listed in Table 1, for the realization of polynomial multiplications required in `hqc-128`, `hqc-192`, the authors of HQC decided to apply a combination of the Toom-Cook multiplication strategy named as Toom-3, which splits each factor in three parts before performing operations on them, with a recursive 3-split Karatsuba multiplication approach or a recursive 5-split Karatsuba multiplication approach, down to the point where a 64-bit×64-bit multiplier is sufficient. For the realization of polynomial multiplications required in `hqc-256` the same overall approach is applied, replacing the 3-split Karatsuba algorithm with a 5-split one. The chosen strategy is supported by a favorable $O(n^{1.46})$ cost of the Toom-Cook approach, which is further optimized, in the finite regime of the HQC parameters, applying also the Karatsuba technique and the (vectorized) multiply instructions provided by the instruction set architecture of the target platform.

Alternate Approach. The asymptotically fastest methods currently known to perform multiplications in $\mathcal{R}$ require to evaluate them at $2n$ points, perform a pointwise multiplication of the these results, and interpolate the resulting $2n$ values to get the final polynomial. While this approach may seem less efficient, with

a proper choice of the evaluation points it takes the name of Fast Fourier Transform (FFT) based multiplication, and the overall computational cost (including the one of interpolation) requires $O(n \cdot \log(n))$ bit operations.

We noted that relatively recent development in the field of FFT-based arithmetic in binary finite fields may allow execution of polynomial multiplications in both $\mathtt{F}_2[x]$ and $\mathcal{R}$ exhibiting a computational complexity that is less than the one that can be obtained with other methods when the bitlength of the operands in in the range prescribed by each HQC instance. Specifically, Cantor [6], developed the so-called additive-FFTs through evaluating polynomials at points that form an additive subgroup in $\mathbb{F}_2$ and defining a basis, termed *Cantor basis*, for constructing an FFT over the field that works with the complexity of $O(n \log(n))$ multiplications and $O(n \cdot \log^{\log_2(3)}(n))$ `xor`-additions for evaluating $n = 2^m$ elements. In 2010, Gao and Mateer [11] presented an additive-FFT (GM-FFT) over $\mathbb{F}_{2^k}$ with $O(n \cdot \log^2(n))$ `xor`-operations for evaluating a polynomial at $n = 2^m$ points in general. It can be optimized to $O(n \cdot \log(n) \cdot \log(\log(n)))$ only when m is a power of 2, and the polynomials in this case are represented in a special basis introduced by Cantor [6]. In 2014, Bernstein and Chou [3] presented an efficient implementation of the GM-FFT with performance equal to the ones obtainable applying the more general variant introduced in the same year by Lin, Chung, and Han [20] (named LCH-FFT), which uses a polynomial basis different from the standard one. In the subsequent work [19] a method to convert from/to the standard polynomial basis to/from the one required by [6] is introduced. The computational complexity to realize the conversion is $O(n \cdot \log(n) \cdot \log(\log(n)))$ `xor`-operations, with n being any power of two, which in turn reaches the same complexity as the common multiplicative FFTs.

We were able to effectively reproduce the multiplication strategy introduced in [3] to the end of realizing the polynomial multiplications listed in the HQC primitives with a constant-time dense-to-dense algorithmic approach. Since the length of the operands n in HQC is not a power of two, a straightforward approach would call for zero-padding of the polynomials for a number of coefficients up to the nearest power of two greater than n. However, this approach performs a significant amount of unnecessary computations among zero-filled words, whenever n is far from the said power of two (as it is the case in HQC-128 and HQC-192). To avoid such overhead, we automatically generated the code of the multiplication algorithm in [3], removing at code generation time the operations acting deterministically only on zero-filled words.[1]

3.4 Boolean Masking of the HQC.PKE against Power Analysis

Regardless of their security against mathematical cryptanalysis, the implementation of cryptographic algorithms on end-user devices may leak information about secret parameters of the scheme, which can lead to successful plaintext or secret-key recovery attacks. Gathering power- or EM-measurements (a.k.a. *traces*) during a known-input execution of a cryptographic primitive allows to

[1] The source code of our HQC realization will be publicly available upon acceptance

apply a divide-and-conquer strategy in inferring the secret values in the computation, thanks to the timewise locality and granularity of the information obtained. The general workflow for non-profiled side channel attacks predicts the side channel behaviour of a simple operation in the cryptographic primitive, depending on a (guessed) value for the small portion of secret involved in it. After collecting measurements from the device, the correct guess for the secret is singled out comparing with statistical means the secret-dependent predictions with the actual measures.

A popular, and quantifiably secure countermeasure against side-channel attacks is *masking* [17,24]. Masking works through performing the entire computation on share-split values, through an x-out-of-x share splitting schemes. The number of shares into which a value is split provides a quantification of the security margin: this is commonly denoted as the *masking order* d, where a masking scheme with x shares is known as a $d = x - 1$ order masking. The most common share splitting strategy is the one known as *Boolean masking*. Applying Boolean masking to a polynomial in $\mathbf{v} \in \mathcal{R}$ we have that the x-uple $[\![\mathbf{v}]\!] = (\mathbf{v}_0, \ldots, \mathbf{v}_{x-1})$ of the shares of a polynomial $\mathbf{v} \in \mathcal{R}$ is obtained drawing $x - 1$ random polynomials, $(\mathtt{rnd}_0, \ldots, \mathtt{rnd}_{x-2})$ and obtaining the remaining share as their coefficient-wise `xor` sum with the polynomial to be split $\mathbf{v}$, $(\mathbf{v} \oplus \bigoplus_{i=0}^{x-2} \mathtt{rnd}_i)$. We denote this operation as $\textsc{Split}(\mathbf{v}) = (\mathbf{v} \oplus \sum_{i=0}^{x-2} \mathtt{rnd}_i, \mathtt{rnd}_0, \ldots, \mathtt{rnd}_{x-2}) = [\![\mathbf{v}]\!]$.

All the operations on share-split polynomials must be performed so that combining the share-split results (i.e., `xor`-ing together all shares) yields the correct result. We will denote such an action on the share-split value $[\![\mathbf{v}]\!]$ as $\textsc{Combine}([\![\mathbf{v}]\!]) = \mathbf{v}$. The share combination is performed only at the end of the primitive computation, as any operation combining all the shares from of a single original value $\mathbf{v}$ reveals information on the value itself. By contrast, combining up to $x - 1$ shares does not yield any information leakage.

Boolean masking incurs in only a linear (in x) overhead whenever the performed operations on share-split polynomials are `xor` linear, nonlinear operations (e.g., Boolean `and`) have a quadratic overhead [17].

To the end of devising our algorithmic proposal for efficient Boolean masking of HQC.PKE, we observe that the following two properties hold for its structure:

Property 1 (Public operands) *All the polynomial multiplications have a factor which is public and either input independent or constant (across a computation). It is thus possible to avoid the share-splitting of the public operand and rewrite the multiplication as a sequence of x share-wise `xor` additions of the share-split operand and x share-wise circular shifts.*

Property 2 (Share-split decoding) *Computing in a share-split representation the decoding w.r.t. the public code $\mathbf{G}$ is performed in a more efficient way exploiting the linearity of the encoding: the $x-1$ random shares for the erroneous codeword to be decoded are obtained encoding $x - 1$ random information words.*

Masked Keygen. The masking of the key generation algorithm (Algorithm 2) is performed splitting the private key ($\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y}) \in \mathcal{R}_\omega \times \mathcal{R}_\omega$) element $\mathbf{v}$ (line

Algorithm 2: Masked HQC.PKE Keygen

Input: none
Output: $\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \in \mathcal{R} \times \mathcal{R}$
$\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y}) \in \mathcal{R}_\omega \times \mathcal{R}_\omega$

1 $\mathbf{h} \xleftarrow{\$} \mathcal{R}$;
2 $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;
3 $[\![\mathbf{y}]\!] \leftarrow \textsc{Split}(\mathbf{y})$;
4 $[\![\mathbf{s}]\!] \leftarrow (\mathbf{h} \cdot \mathbf{y}_{,0}, \ldots, \mathbf{h} \cdot \mathbf{y}_{,x-1})$;
5 $[\![\mathbf{x}]\!] \leftarrow \textsc{Split}(\mathbf{x})$;
6 $[\![\mathbf{s}]\!] \leftarrow [\![\mathbf{x}]\!] \oplus [\![\mathbf{s}]\!]$;
7 $\mathcal{K}_{pri} \leftarrow (\mathbf{x}, \mathbf{y})$;
8 $\mathcal{K}_{pub} \leftarrow (\mathbf{h}, \textsc{Combine}([\![\mathbf{s}]\!]))$;
9 **return** $\mathcal{K}_{pri}$, $\mathcal{K}_{pub}$;

Algorithm 3: Masked HQC.PKE Encrypt

Data: $\mathbf{G}$: a $\mathbb{F}_2^{k \times n}$ matrix generating an efficiently correctable code
Input: $\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \in \mathcal{R} \times \mathcal{R}$
$\mathbf{m} \in \mathbb{F}_2^k$
Output: $(\mathbf{u}, \mathbf{v}) \in \mathcal{R} \times \mathcal{R}$

1 $\mathbf{e} \xleftarrow{\$} \mathcal{R}_{\omega_e}$, $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;
2 $[\![\mathbf{r_1}]\!] \leftarrow \textsc{Split}(\mathbf{r_1})$;
3 $[\![\mathbf{r_2}]\!] \leftarrow \textsc{Split}(\mathbf{r_2})$;
4 $[\![\mathbf{u}]\!] \leftarrow [\![\mathbf{r_1}]\!] \oplus (\mathbf{h} \cdot \mathbf{r_2}_{,0}, \ldots, \mathbf{h} \cdot \mathbf{r_2}_{,x-1})$;
5 $[\![\mathbf{m}]\!] \leftarrow \textsc{Split}(\mathbf{m})$;
6 $[\![\mathbf{v}]\!] \leftarrow [\![\mathbf{m}]\!]\mathbf{G} \oplus \textsc{Split}(\mathbf{e})$;
7 $[\![\mathbf{v}]\!] \leftarrow [\![\mathbf{v}]\!] \oplus (\mathbf{s} \cdot \mathbf{r_2}_{,0}, \ldots, \mathbf{s} \cdot \mathbf{r_2}_{,x-1})$;
8 $\mathbf{u} \leftarrow \textsc{Combine}([\![\mathbf{u}]\!])$;
9 $\mathbf{v} \leftarrow \textsc{Combine}([\![\mathbf{v}]\!])$;
10 **return** $(\mathbf{u}, \mathbf{v})$;

3), and multiplying it by the public polynomial $\mathbf{h}$, which does not need to be share-split (Property 1). To add the secret value $\mathbf{x}$ to $[\![\mathbf{s}]\!]$, we share-split it (line 5) in turn and perform a share-wise `xor`. Finally, we combine the shares of $[\![\mathbf{s}]\!]$ to obtain the second half of the public key.

Masked Encrypt. In the encryption algorithm, we protect both the message $\mathbf{m}$ and the ephemeral values $\mathbf{r_1}, \mathbf{r_2}, \mathbf{e}$ by share splitting them (lines $2, 3, 5, 6$). The multiplications at lines 4 and 6 can be performed without share splitting the public factors $\mathbf{h}$ and $\mathbf{s}$ – by Property 1, indeed, they are the two components of the public key – and therefore their share-wise multiplication only takes linear time in the number of shares x. The encoding of the shared message $[\![\mathbf{m}]\!]$ is performed via a sequence of linear operations, i.e. the share-wise multiplication by the public binary matrix $\mathbf{G}$, which therefore does not need to be share split, or any equivalent encoding technique.

Masked Decrypt. The decryption function needs to protect the private key component $\mathbf{y}$, which is split (line 1 of Algorithm 4), and subsequently multiplied by the public ciphertext component $\mathbf{u}$, which does not need splitting (Property 1). We now use Property 2 to protect the decoding procedure (line 6) without computing x times the decoding algorithm, which is the most demanding portion of HQC.PKE Decrypt. To this end, we denote with a helper procedure, $\textsc{Collapse}(\mathbf{a}, [\![\mathbf{b}]\!])$ the `xor` sum of $\mathbf{a}$ with all the shares of $\mathbf{b}$, done in storage order of the shares themselves, as doing a hierarchical reduction would lower the number of side channel samples to know for an attacker [17,24]. We protect the input of the Decode procedure, adding a random codeword to it. This random codeword is obtained drawing a random information word $\mathbf{b}$ (line 4), share splitting it (line 5) and encoding all the shares, obtaining the share-split codeword $[\![\mathbf{c}]\!]$ (line 6). We note that a single Decode computation is performed, as opposed

Algorithm 4: Masked HQC.PKE Decrypt

Data: $\mathbf{G}$: a $\mathbb{F}_2^{k\times n}$ matrix generating an efficiently correctable code
Input: $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y}) \in \mathcal{R}_\omega \times \mathcal{R}_\omega$
$(\mathbf{u}, \mathbf{v}) \in \mathcal{R} \times \mathcal{R}$
Output: $[\![\mathbf{b}]\!]$, such that $\textsc{Combine}([\![\mathbf{b}]\!]) = \mathbf{m} \in \mathbb{F}_2^k$

1 $[\![\mathbf{y}]\!] \leftarrow \textsc{Split}(\mathbf{y})$;
2 $[\![\mathbf{uy}]\!] \leftarrow (\mathbf{u}\cdot\mathbf{y}_0, \ldots, \mathbf{u}\cdot\mathbf{y}_{x-1})$;
3 $\mathbf{b} \xleftarrow{\$} \mathbb{F}_2^k$;
4 $[\![\mathbf{b}]\!] \xleftarrow{\$} \textsc{Split}(\mathbf{b})$;
5 $[\![\mathbf{c}]\!] \leftarrow [\![\mathbf{b}]\!]\mathbf{G}$;
6 $\mathbf{a} \leftarrow \textsc{Collapse}(\mathbf{v}, [\![\mathbf{c}]\!])$;
7 $\overline{\mathbf{b}} \leftarrow \textsc{Decode}(\textsc{Collapse}(\mathbf{a}, [\![\mathbf{uy}]\!]))$; // $\overline{\mathbf{b}} = \mathbf{m} \oplus \mathbf{b}$
8 $\mathbf{b}_0 \leftarrow \overline{\mathbf{b}} \oplus \mathbf{b}_0$; // this sets $[\![\mathbf{b}]\!] = (\mathbf{m} \oplus \mathbf{b} \oplus \mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{x-1})$
9 **return** $[\![\mathbf{b}]\!]$;

to a straightforward masking scheme which would require x of them to be computed. Thanks to Property 2, decoding the sum of share-split codewords will result in the sum of the corresponding share-split information words. We therefore collapse the share-split codeword $[\![\mathbf{c}]\!]$ onto the public decoder input, $\mathbf{v}$, while also performing the subtraction $\mathbf{v} - \mathbf{uy}$ (lines 6,7, recalling that the subtraction among binary polynomials amounts to a coefficient-wise `xor`). The result of the Decode procedure $\mathbf{b}_0$ is thus the message information word $\mathbf{m}$, with a random information word $\mathbf{b}$ added to it. Adding such a value to the first share of the share-split representation of $[\![\mathbf{b}]\!] = (\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{x-1})$ turns such a representation into a share split representation of $\mathbf{m} = (\mathbf{m} \oplus \mathbf{b} \oplus \mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{x-1})$ which is the value returned by the protected HQC.PKE Decrypt. Such a value would be recombined, in case the HQC.PKE were used alone; instead, considering the HQC.KEM, can be forwarded to the protected components of the FO construction for further processing.

4 Experimental Evaluation

In this section we report the results of the efficiency and effectiveness of our proposed masked implementation. To this end, we choose the Nucleo L476RG board, which is endowed with a STM32L476RG Cortex-M4 microcontroller (which can be clocked up to 80MHz), 128kiB of RAM, 1MiB of Flash memory, and a true random number generator. The choice of the platform was made as it is part of the `pqm4` benchmarking project [18], allowing us fair comparisons with the current state of the art, and the chosen microcontroller is the one picked by NIST as reference platform. The available memory on the microcontroller constrains us to the `hqc-128` parameter set alone and a $x = 2$ out of 2 share splitting, as larger ones have operands which do not fit, once share-split, into the RAM of the microcontroller. We employed `libopencm3` 1.0.0 as our hardware abstraction layer library and compiled our code with `gcc-arm-none-eabi` 7.2.1 using

Table 2: t-values for the HQC.KEM algorithms. The '•' in the **CT?** column indicates that the multiplication algorithm employed is constant-time

NIST Round	Mult. Alg.	CT?	Encrypt	Decrypt	Encaps	Decaps
3	Shift&Add	○	29.19	39.23	28.65	18.44
	CT Shift&Add	•	23.12	49.56	14.29	23.70
4	Karatsuba-based	•	1.04	0.30	4.01	0.26
	CT Shift&Add	•	0.07	1.64	2.17	0.22
	additive-FFT	•	0.56	1.99	4.25	0.53

the `-O3` optimization flag. We performed our evaluation clocking the microcontroller with frequencies in the $\{16, 32, 48, 64, 80\}$MHz set to elicit possible timing vulnerabilities due to different Flash memory wait states.

Side-channel Security Evaluation. To detect the presence of an information leakage on either the timing or the power consumption side channel we employ the well-established Test Vector Leakage Assessment (TVLA) [29] procedure, based on Welch's t-test. The technique collects two sample populations, one from the cryptographic algorithm running on a fixed value as an input, the other running on randomly generated inputs, while the cryptographic-key in both cases is a fixed value. TVLA subsequently employs a Student's t-test built with the null-hypothesis that the populations have different means (i.e., some information leakage takes place). The widely accepted bar for the t-test to reject the null-hypothesis and hence indicate absence of side channel leakage is that the value of the t statistic is in the range $[-4.5; 4.5]$. Small t values give (statistical) evidence to reject the null hypothesis and, as a consequence, to be assured that the two populations have the same mean. Specifically, values of $t \in [-4.5; 4.5]$ allow, with a confidence level of 99.999%, the null hypothesis to be rejected, and then to be highly confident that the side-channel behavior of the target implementation fed with random plaintexts cannot be distinguished from the one obtained running the target implementation with a fixed plaintext.

To measure the execution times we use the *Data Watchpoint and Trace Cycle Count Register* (`DWT_CYCCNT`), a 32-bit register that counts the number of clock cycles elapsed since its last reset, with a resolution of one clock cycle; to avoid overflows, we reset the register after all the measurements taken from each execution of the target implementation. Table 2 reports the values of the t statistic for the implementations of the entire HQC.KEM as provided by the authors to NIST for both Round 3 and Round 4 of the competition. For the sake of evaluation and comparison we tested their implementations also replacing our multiplication algorithms based on a constant time shift and add strategy as well as on the additive-FFT. The t values reported in Table 2 have been obtained gathering $1,000$ samples for each population (i.e., fixed and randomized plaintexts), and setting the microcontroller frequency to 80MHz. The obtained results are analogous picking any working frequency in the in the $\{16, 32, 48, 64, 80\}$MHz set, and are omitted for lack of space. We note that reporting a pass in the t-test for the

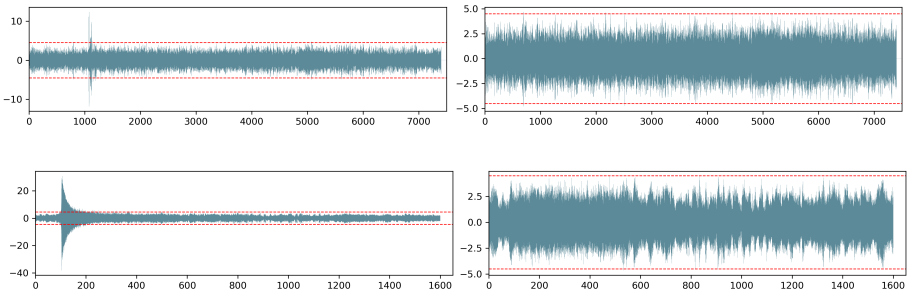


Fig. 1: t-value statistics on the electromagnetic traces on the protected RM-RS decoder (top), and zoom on the RS decoder alone (bottom). Figures on the left are from the unprotected implementations, figures on the right from the protected one. All results obtained with 50k traces.

entire HQC.KEM is useful to confirm that the entire implementation is constant time. Indeed, Guo et. al [14] highlighted that also components besides the multipliers may expose timing information leakage. This fact is highlighted by both implementations of the Round 3 HQC proposal having reported t statistics outside the acceptable interval $[-4.5; 4.5]$ (e.g., 23.12 from the execution of even the ENCRYPT primitive alone, despite the use of a constant time multiplication algorithm). For the one employing the variable-time shift-and-add multiplication the effect is more evident, while the one employing a constant-time multiplication algorithm has its timing leakage stemming from the sparse polynomial sampler proposed for the Round 3 HQC.KEM implementation, as reported in [14]. All Round 4 HQC.KEM implementations, employing a Karatsuba-based multiplication strategy (as proposed by the HQC authors), or either a constant-time (CT) shift-and-add, or additive-FFT multiplication algorithm (proposed by us) have values for the t statistic within the required interval.

We employ the electromagnetic emissions as a proxy of the power consumption, and measure them with a PicoScope 5244D oscilloscope sampling at 500MS/s the output of an H-probe placed on top of the STM32L476RG package, amplifying the signal with two Agilent INA-10386 amplifiers a 52dB gain. We collect 50k samples for each population, and run the t test timewise, as per the TVLA [29] methodology, obtaining time sequences of t-statistics. Figure 1 reports plots of the t statistic as a function of time for both the unprotected (left) and protected (right) implementations. In particular, the topmost two figures correspond to the computation of the RMRS decoder, while the bottom ones are a zoom on the RS decoder alone. The spike in the value of the t statistic, reaching well beyond the admissible limit for the unprotected implementation corresponds, in time to the calculation of the error corrector polynomial, which was also reported by [27] as an information leaking computation. By contrast, the time series of the protected implementation are within the $[-4.5; 4.5]$ interval, depicted by dashed red lines, showing the effectiveness of our countermeasure.

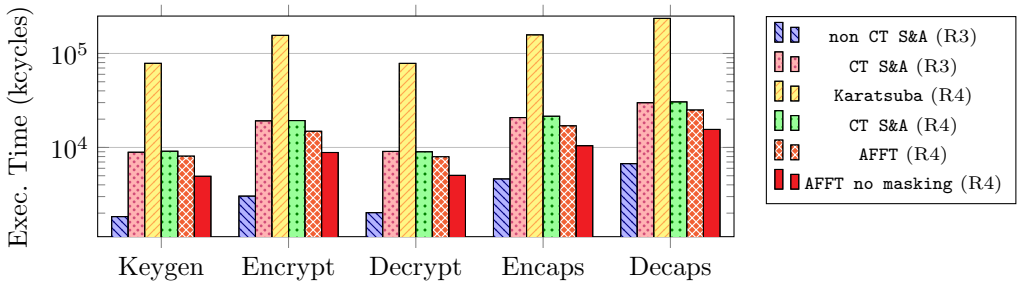


Fig. 2: Execution times of the encryption, decryption, encapsulation and decapsulation algorithms of the `HQC.KEM` on the `ARM` architecture. The measured components of the `HQC.KEM` include both the non-constant time sampler from Round 3 (R3) and the constant time from Round 4 (R4). `AFFT` marks the additive-FFT multiplier based implementations

Efficiency evaluation. We evalutated the computational efficiency of all the considered multiplication approaches reported in Table 2. Figure 2 reports the number of clock cycles taken by a full execution of the HQC.KEM algorithms, considering their masked implementation (depicted with patterned columns), and provide the results of the unprotected implementation only for the most efficient, fully constant time proposal (i.e. the one based on the Round 4 sampler and FFT-based multipliers). The results in Figure 2 show how the FFT-multiplier based HQC.KEM is almost an order of magnitude faster than the corresponding Karatsuba multiplier based one (note that the plot is in logarithmic scale on the y axis). Our FFT-based multiplier approach also consistently outperforms the constant-time shift-and-add one, albeit by a smaller margin. Finally, we note that the performance overhead which our algorithmic approach to compute the HQC.KEM as masked is consistently within the $2\times$ range which is expected from its sub-linear cost in the number of shares (two, in the figures we report). Consistently with [15], we also report that the main performance overhead comes from the additional multiplications, the impact of the masking on the hash function is negligible with respect to the total computation time.

5 Concluding Remarks

In this work, we presented the first constant-time and fully masked implementation of the full HQC.KEM, tailored to Cortex-M4 microcontrollers, exploring the possible constant time multiplication techniques. We proposed a masking approach to protect the entire HQC.KEM against power consumption side channel attacks with a cost lower than linear in the number of shares employed in the masking. Our results shows the practicality of employing a fully masked HQC on microcontrollers, hafing performance which are competitive with the other Round 4 NIST PQC candidates: indeed our HQC masked decapsulation is faster than the one for a recent unprotected implementation of BIKE [7] (full comparison with Round 4 candidates in the Appendix at Figure 3).

References

1. Alekhnovich, M.: More on Average Case vs Approximation Complexity. In: 44th Symposium on Foundations of Computer Science (FOCS 2003), 11-14 October 2003, Cambridge, MA, USA, Proceedings. pp. 298–307. IEEE CS (2003). https://doi.org/10.1109/SFCS.2003.1238204
2. Aragon, N., Barreto, P., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Gueron, S., Guneysu, T., Melchor, C.A., Misoczki, R., Persichetti, E., Sendrier, N., Tillich, J.P., Zemor, G., Vasseur, V., Ghosh, S., Richter-Brokmann, J.: BIKE Supporting Documentation. https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.10.1.pdf (2022)
3. Bernstein, D.J., Chou, T.: Faster Binary-Field Multiplication and Faster Binary-Field MACs. In: Joux, A., Youssef, A.M. (eds.) Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14-15, 2014, Revised Selected Papers. LNCS, vol. 8781, pp. 92–111. Springer (2014). https://doi.org/10.1007/978-3-319-13051-4_6
4. Bernstein, D.J., Chou, T., Cid, C., Gilcher, J., Lange, T., Maram, V., von Maurich, I., Misoczki, R., Niederhagen, R., Persichetti, E., Peters, C., Sendrier, N., Szefer, J., Tjhai, C.J., Tomlinson, M., Wang, W.: Classic McEliece Supporting Documentation. https://classic.mceliece.org/mceliece-spec-20221023.pdf (2022)
5. Biondi, M., Heid, A.: Quantum computing use cases are getting real—what you need to know. A burgeoning quantum-computing ecosystem and emerging business use cases promise to create significant value for industries—if executives prepare now. McKinsey's Report - Dec. 2021, https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/quantum-computing-use-cases-are-getting-real-what-you-need-to-know (2021)
6. Cantor, D.G.: On arithmetical algorithms over finite fields. J. Comb. Theory, Ser. A **50**(2), 285–300 (1989). https://doi.org/10.1016/0097-3165(89)90020-4
7. Chen, M., Chou, T., Krausz, M.: Optimizing BIKE for the intel haswell and ARM cortex-m4. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2021**(3), 97–124 (2021). https://doi.org/10.46586/tches.v2021.i3.97-124
8. Dworkin, M.: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Federal Inf. Process. Stds. 202 (NIST FIPS.202), National Institute of Standards and Technology, Gaithersburg, MD (2015-08-04 2015). https://doi.org/https://doi.org/10.6028/NIST.FIPS.202
9. Fujisaki, E., Okamoto, T.: Secure Integration of Asymmetric and Symmetric Encryption Schemes. In: Wiener, M.J. (ed.) Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings. LNCS, vol. 1666, pp. 537–554. Springer (1999). https://doi.org/10.1007/3-540-48405-1_34
10. Gamal, T.E.: A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Trans. Inf. Theory **31**(4), 469–472 (1985). https://doi.org/10.1109/TIT.1985.1057074
11. Gao, S., Mateer, T.D.: Additive Fast Fourier Transforms Over Finite Fields. IEEE Trans. Inf. Theory **56**(12), 6265–6272 (2010). https://doi.org/10.1109/TIT.2010.2079016
12. Goy, G., Loiseau, A., Gaborit, P.: A New Key Recovery Side-Channel Attack on HQC with Chosen Ciphertext. In: Cheon, J.H., Johansson, T. (eds.) Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings. LNCS, vol. 13512, pp. 353–371. Springer (2022). https://doi.org/10.1007/978-3-031-17234-2_17

13. Goy, G., Loiseau, A., Gaborit2, P.: Estimating the Strength of Horizontal Correlation Attacks in the Hamming Weight Leakage Model: A Side-Channel Analysis on HQC KEM. In: WCC 2022: The Twelfth International Workshop on Coding and Cryptography March 7 - 11, 2022, Rostock (Germany) (2022), https://www.wcc2022.uni-rostock.de/storages/uni-rostock/Tagungen/WCC2022/Papers/WCC_2022_paper_48.pdf
14. Guo, Q., Hlauschek, C., Johansson, T., Lahr, N., Nilsson, A., Schröder, R.L.: Don't Reject This: Key-Recovery Timing Attacks Due to Rejection-Sampling in HQC and BIKE. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2022**(3), 223–263 (2022). https://doi.org/10.46586/tches.v2022.i3.223-263
15. Heinz, D., Kannwischer, M.J., Land, G., Pöppelmann, T., Schwabe, P., Sprenkels, A.: First-Order Masked Kyber on ARM Cortex-M4. IACR Cryptol. ePrint Arch. p. 58 (2022), https://eprint.iacr.org/2022/058
16. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A Modular Analysis of the Fujisaki-Okamoto Transformation. In: Kalai, Y., Reyzin, L. (eds.) Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I. LNCS, vol. 10677, pp. 341–371. Springer (2017). https://doi.org/10.1007/978-3-319-70500-2_12
17. Ishai, Y., Sahai, A., Wagner, D.A.: Private Circuits: Securing Hardware against Probing Attacks. In: Boneh, D. (ed.) Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings. LNCS, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27
18. Kannwischer, M.J., Petri, R., Rijneveld, J., Schwabe, P., Stoffelen, K.: PQM4: Post-quantum crypto library for the ARM Cortex-M4, https://github.com/mupq/pqm4
19. Lin, S., Al-Naffouri, T.Y., Han, Y.S.: FFT algorithm for binary extension finite fields and its application to reed-solomon codes. IEEE Trans. Inf. Theory **62**(10), 5343–5358 (2016). https://doi.org/10.1109/TIT.2016.2600417
20. Lin, S., Chung, W., Han, Y.S.: Novel Polynomial Basis and Its Application to Reed-Solomon Erasure Codes. In: 55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014. pp. 316–325. IEEE CS (2014). https://doi.org/10.1109/FOCS.2014.41
21. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehle, D., Bai, S.: Kyber Supporting Documentation. https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf (2021)
22. Melchor, C.A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Persichetti, E., Zémor, G., Bos, J., Dion, A., Lacan, J., Robert, J.M., and, P.V.: HQC Supporting Documentation. http://pqc-hqc.org/doc/hqc-specification_2023-04-30.pdf (2023)
23. Melchor, C.A., Blazy, O., Deneuville, J., Gaborit, P., Zémor, G.: Efficient Encryption From Random Quasi-Cyclic Codes. IEEE Trans. Inf. Theory **64**(5), 3927–3943 (2018). https://doi.org/10.1109/TIT.2018.2804444
24. Prouff, E., Rivain, M.: Masking against Side-Channel Attacks: A Formal Security Proof. In: Johansson, T., Nguyen, P.Q. (eds.) Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings. LNCS, vol. 7881, pp. 142–159. Springer (2013). https://doi.org/10.1007/978-3-642-38348-9_9
25. Rivest, R.L., Shamir, A., Adleman, L.M.: A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. Commun. ACM **21**(2), 120–126 (1978). https://doi.org/10.1145/359340.359342

26. Schamberger, T., Holzbaur, L., Renner, J., Wachter-Zeh, A., Sigl, G.: A Power Side-Channel Attack on the Reed-Muller Reed-Solomon Version of the HQC Cryptosystem. In: Cheon, J.H., Johansson, T. (eds.) Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings. LNCS, vol. 13512, pp. 327–352. Springer (2022). https://doi.org/10.1007/978-3-031-17234-2_16
27. Schamberger, T., Holzbaur, L., Renner, J., Wachter-Zeh, A., Sigl, G.: A Power Side-Channel Attack on the Reed-Muller Reed-Solomon Version of the HQC Cryptosystem. In: Cheon, J.H., Johansson, T. (eds.) Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings. LNCS, vol. 13512, pp. 327–352. Springer (2022). https://doi.org/10.1007/978-3-031-17234-2_16
28. Schamberger, T., Renner, J., Sigl, G., Wachter-Zeh, A.: A Power Side-Channel Attack on the CCA2-Secure HQC KEM. In: Liardet, P., Mentens, N. (eds.) Smart Card Research and Advanced Applications - 19th International Conference, CARDIS 2020, Virtual Event, November 18-19, 2020, Revised Selected Papers. LNCS, vol. 12609, pp. 119–134. Springer (2020). https://doi.org/10.1007/978-3-030-68487-7_8
29. Schneider, T., Moradi, A.: Leakage assessment methodology - extended version. J. Cryptogr. Eng. **6**(2), 85–99 (2016). https://doi.org/10.1007/s13389-016-0120-y
30. Shor, P.W.: Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In: 35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994. pp. 124–134. IEEE CS (1994). https://doi.org/10.1109/SFCS.1994.365700

Appendix

Algorithm 5: Sparse-to-dense Polynomial Multiplication

Input: n: maximum number of bits for a polynomial in $\mathcal{R}=\mathbb{F}_2[x]/(x^n-1)$.
`wsize`: bitlength of a CPU word, $\mathtt{w} = 2^{\mathtt{wsize}}$ radix employed to represent in positional notation a long integer/binary-polynomial.
$A = (A_{\omega-1}, \ldots, A_i, \ldots, A_0)$, represents $a(x) \in \mathcal{R}$, where A_i is an unsigned integer that encodes the exponent of a non-null monomial $x^a \in \mathcal{R}$, $0 \leq a \leq n-1$, $\omega < n$.
$B = (B_{t-1}, \ldots, B_j, \ldots, B_0)_{\mathtt{w}}$, $t = \lceil \frac{n}{\mathtt{wsize}} \rceil$, represents $b(x) \in \mathcal{R}$, where a limb B_j stores a sequence of binary coefficients fitting in a CPU word having `wsize` bits (i.e., $0 \leq B_j < \mathtt{w}$), $0 \leq j \leq t-1$.

Output: $C = (C_{t-1}, \ldots, C_k, \ldots, C_0)_{\mathtt{w}}$, represents $c(x) = a(x) \cdot b(x) \in \mathcal{R}$, where each limb C_k stores a sequence of binary coefficients fitting into a CPU word (i.e., $0 \leq C_k < \mathtt{w}$), $0 \leq k \leq t-1$.

$C \leftarrow (0, \ldots, 0)$; // zeroed array with t cells, each with *wsize* bits
for $i \leftarrow 0$ **to** $\omega - 1$ **do**
  $\mathtt{tmp} \leftarrow (\ B \lll_n A_i\ )$; // variable-time comput. of a t-word array
  for $k \leftarrow 0$ **to** $(t-1)$ **do**
    $C_k \leftarrow C_k \oplus \mathtt{tmp}_k$;
return C;

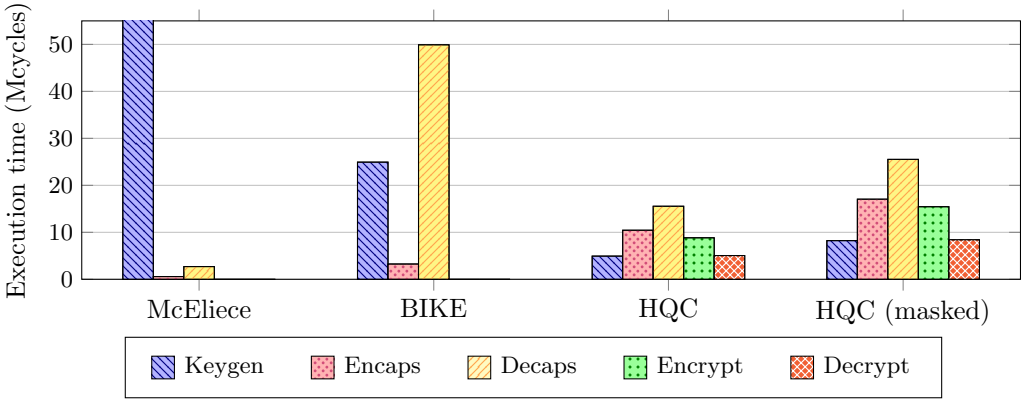


Fig. 3: Comparison of the execution time for all Round 4 standardization candidates: Classic McEliece, BIKE, and HQC. All timings collected on a STM32L476RG Cortex-M4 running at 80MHz employing the optimized implementations available in [18], save for our HQC implementation. All implementations unprotected unless noted otherwise.